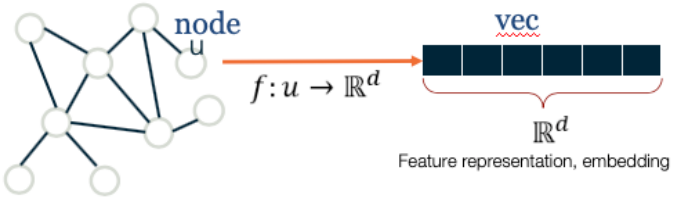
L14: Embedding Nodes in a Low-dimensional Space

Machine Learning (ML) is a mature discipline with powerful methods to solve classification, clustering, regression and many other problems in data science. Most ML methods however require a *vector representation of the inputs.* This is easy to do with structured data that already come in the form of vectors or matrices. Even in the case of images, we can easily create a vector of pixel intensities (or RGB colors), scanning the image row by row.

How can we represent a graph (or network), however, with a vector? A graph can have an arbitrary structure, with some nodes being much more connected than others.

One option would be to represent each node *v* with an *N*-dimensional binary vector, where *N* is the number of nodes: element *i* is one if node-i is a neighbor of node v and zero otherwise. That vector representation however would be of high dimensionality for large networks. The *sampling efficiency* of most ML algorithms (i.e., how much training data they need in order to learn a model) is determined by that dimensionality. So, we are interested in graph representation methods that will map each node to a *d*-dimensionality vector, with $d \ll N$. The *d*-dimensional vector that corresponds to a node *u* is referred to as the *"embedding"* ***f(u)*** of that node (see Figure below).



Images (a,b) from Jure Leskovec , “Representation Learning on Networks” tutorial, WWW 2018

Consider, for instance, the well-known Zachary's karate club network (we had also examined that network in Lesson-7), and suppose that we want to represent each node with a two-dimensional vector, i.e., with a point in the Euclidean plane. How would you choose those *N* points?

One approach would be to map *any two nodes that are "close to each other"* in terms of network distance (e.g., directly connected with each other, or having a relatively large number of common neighbors) to *two nearby points in the plane*. And similarly, any two nodes that have a large network distance should be mapped to points with relatively large distance in the plane. The following figure shows a particular embedding of the nodes in Zachary's network using the *DeepWalk algorithm*, which will be covered later in this Lesson.

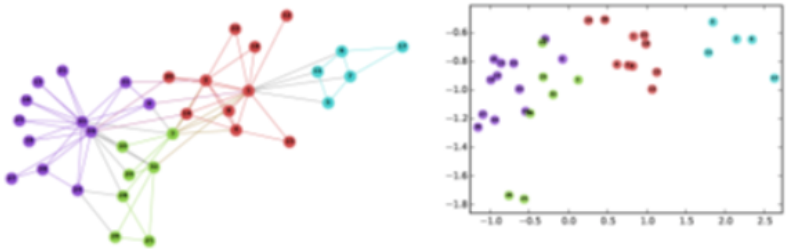


Image from: **Perozzi et al. 2014** (https://arxiv.org/pdf/1403.6652.pdf) . DeepWalk: Online Learning of Social Representations. *KDD.*

Why is it reasonable to create node embeddings based on the network distance between two nodes? The basic idea is that the network distance between two nodes in a graph is usually related to the similarity between those two nodes. For instance, in the context of social networks, two individuals that have strong ties between them and/or many common friends, would typically be similar in terms of interests. At the same time, many ML methods (such as classifiers based on the *K-nearest neighbors*

algorithm or based on *Support Vector Machines)* are based on the assumption that the similarity between two inputs is reflected by the distance between their two embeddings.

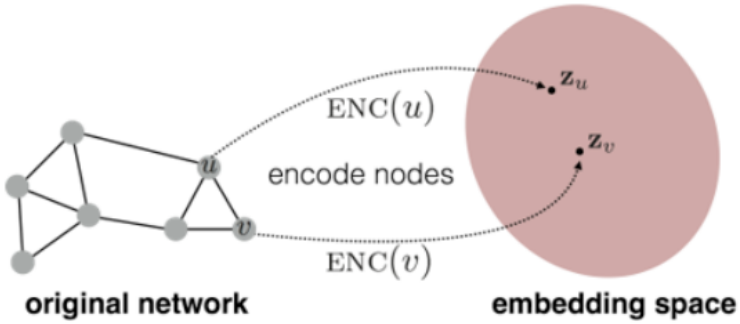


Figure-b shows the original network and embedding space.

For example, if our goal is to classify "politically unlabeled" people in a social network (say whether they are Democrats, Republicans, Independent, etc), the idea of creating embeddings based on the network connectedness of two nodes makes a lot of sense: individuals that belong in the same social groups will be mapped to nearby embeddings.

Another example would be in the context of community detection. If the node embeddings are based on network distance, we can use any ML clustering algorithm (such as *K-means*) to identify clusters of nearby points in the *d*-dimensional embedding space.

As you can imagine, there are many way to compute embeddings from a graph so that well-connected nodes are mapped to nearby (or similar) vectors. In the next few pages we will examine more closely some popular methods to compute node embeddings.

Food For Thought

The Karate club embeddings are deliberately shown here in low-resolution so that the label of the nodes is not clear. Can you identify at least five of the nodes from their embeddings?